

Python Sets

What is a Set?



- **A set is an unordered, mutable collection of unique elements in Python.**
- **Key Characteristics:**
 - Unordered: No index-based access; elements have no fixed position
 - Mutable: Can add and remove elements after creation
 - Unique: Automatically removes duplicate values
 - Hashable elements only: Can only store immutable types (int, str, tuple)
- **Sets are written with curly braces { } or the set() constructor:**

```
my_set = {1, 2, 3}
my_set = set([1, 2, 3])
```

Creating Sets



- **Empty Set (IMPORTANT: {} creates a dict, not a set!):**

```
empty = set()          # Correct way to create empty set
not_a_set = {}         # This creates an empty DICTIONARY!
```

- **From a List (removes duplicates automatically):**

```
nums = set([1, 2, 2, 3, 3, 3])
# nums → {1, 2, 3}
```

- **From a String (each character becomes an element):**

```
chars = set('hello')
# chars → {'h', 'e', 'l', 'o'}
```

Creating Sets



- **From a Tuple:**

```
t = set((1, 2, 3, 3))
```

```
# t → {1, 2, 3}
```

- **Set Comprehension:**

```
squares = {x**2 for x in range(5)}
```

```
# squares → {0, 1, 4, 9, 16}
```

Adding & Removing Elements



- **Adding Elements:**

```
s = {1, 2, 3}
```

```
s.add(4)           # s → {1, 2, 3, 4}
```

```
s.add(2)           # No error, but 2 already exists – ignored
```

```
s.update([5, 6])    # Add multiple elements
```

```
s.update({7, 8})    # Can also add from another set
```

```
s.update((9, 10))   # Or from a tuple
```

- **Removing Elements:**

```
s.remove(3)         # Removes 3; KeyError if 3 not present
```

```
s.discard(3)        # Removes 3 if present; NO error if absent
```

```
s.pop()             # Removes and returns an arbitrary element
```

```
s.clear()           # Removes all elements; s becomes set()
```

Set Operators: Union & Intersection



- **Union ($A \cup B$): All elements from both sets (no duplicates)**

Method: `A.union(B)` $\rightarrow \{1, 2, 3, 4, 5\}$

Operator: `A | B` $\rightarrow \{1, 2, 3, 4, 5\}$

Example:

`A = {1, 2, 3}`

`B = {3, 4, 5}`

`A | B` $\rightarrow \{1, 2, 3, 4, 5\}$

Set Operators: Union & Intersection



- **Intersection ($A \cap B$): Common elements in both sets**

Method: `A.intersection(B)` $\rightarrow \{3\}$

Operator: `A & B` $\rightarrow \{3\}$

`A = {1, 2, 3}`

`B = {3, 4, 5}`

`A & B` $\rightarrow$ `{3}`

Set Operators: Difference & Symmetric Difference



- **Difference (A - B): Elements in A but NOT in B**

Method: `A.difference(B)` $\rightarrow \{1, 2\}$

Operator: `A - B` $\rightarrow \{1, 2\}$

`A = {1, 2, 3}`

`B = {3, 4, 5}`

`A - B` $\rightarrow$ `{1, 2}`

`B - A` $\rightarrow$ `{4, 5}` (Note: NOT commutative!)

Set Operators: Difference & Symmetric Difference



- **Symmetric Difference ($A \triangle B$): Elements in either A or B, but NOT both**

Method: `A.symmetric_difference(B)` $\rightarrow \{1, 2, 4, 5\}$

Operator: `A ^ B` $\rightarrow \{1, 2, 4, 5\}$

`A = {1, 2, 3}`

`B = {3, 4, 5}`

`A ^ B` $\rightarrow \{1, 2, 4, 5\}$

Set Comparison & Subset/Superset



- **Subset ($A \subseteq B$):** Is every element of A in B?

Method:	<code>A.issubset(B)</code>	→ True / False
Operator:	<code>A <= B</code>	→ True / False
Proper:	<code>A < B</code>	→ True if $A \subset B$ and $A \neq B$

`A = {1, 2}`

`B = {1, 2, 3, 4}`

`A <= B` → True

`A < B` → True

Set Comparison & Subset/Superset



- **Superset ($A \supseteq B$): Is every element of B in A?**

Method:	<code>A.issuperset(B)</code>	→ True / False
Operator:	<code>A >= B</code>	→ True / False
Proper:	<code>A > B</code>	→ True if $A \supset B$ and $A \neq B$

- **Disjoint: Do A and B have NO elements in common?**

`A.isdisjoint(B)` → True / False

Set Methods: Update Operations



- **These methods modify the set (return None):**

```
s.update(iterable)          # Add all elements from iterable  
s |= other_set              # Same as update, using operator
```

```
s.intersection_update(other) # Keep only common elements  
s &= other_set               # Same, using operator
```

```
s.difference_update(other)   # Remove elements found in other  
s -= other_set               # Same, using operator
```

```
s.symmetric_difference_update(other)  
s ^= other_set               # Keep elements in either set, not both
```

Set Methods: Update Operations



Example:

```
s = {1, 2, 3, 4}
s.intersection_update({3, 4, 5, 6})
# s is now {3, 4}
```

Built-in Functions with Sets



- **len(s)** → Returns the number of elements

`len({1, 2, 3, 3})` → 3

- **max(s) / min(s)** → Returns max/min element

`max({10, 50, 20})` → 50

`min({10, 50, 20})` → 10

- **sum(s)** → Returns the sum of all numeric elements

`sum({1, 2, 3, 4})` → 10

- **sorted(s)** → Returns a sorted LIST (set is unordered!)

`sorted({3, 1, 4, 1})` → [1, 3, 4]

Built-in Functions with Sets



- **set(iterable)** → Converts iterable to a set

`set([1, 2, 2, 3])` → `{1, 2, 3}`

`set('hello')` → `{'h', 'e', 'l', 'o'}`

- **enumerate(s)** → Returns index-value pairs (arbitrary order!)

`list(enumerate({'a', 'b'}))` → `[(0, 'a'), (1, 'b')]` (order varies)

Set Comprehensions



- **A concise way to create sets from iterables:**

Syntax: {expression for item in iterable if condition}

- **Examples:**

Squares of even numbers from 0 to 9

```
{x**2 for x in range(10) if x % 2 == 0}
```

→ {0, 4, 16, 36, 64}

- **Set comprehensions automatically handle duplicates!**

Frozen Sets



- A frozenset is an IMMUTABLE version of a set.
- Once created, you cannot add or remove elements.

- **Creation:**

```
fs = frozenset([1, 2, 3, 3])  
# fs → frozenset({1, 2, 3})
```

- **All set operations work (union, intersection, etc.):**

```
fs | {3, 4, 5} → frozenset({1, 2, 3, 4, 5})
```

- **But NO in-place modification methods:**

```
fs.add(4)          # AttributeError!  
fs.remove(1)       # AttributeError!
```

Set vs List vs Tuple



Feature	Set	List	Tuple
Syntax	{1, 2, 3}	[1, 2, 3]	(1, 2, 3)
Ordered?	No	Yes	Yes
Mutable?	Yes	Yes	No
Duplicates?	No	Yes	Yes
Indexing?	No	Yes	Yes
Use as dict key	No	No	Yes
Set operations	Yes	No	No
Memory	Efficient	Moderate	Efficient

- **Sets are optimized for fast membership testing and set operations.**

Practical Applications



- **Removing Duplicates from a List:**

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique_nums = list(set(nums))
# → [1, 2, 3, 4] (order may vary!)
```

- **Finding Common Elements:**

```
students_A = {'Alice', 'Bob', 'Charlie'}
students_B = {'Bob', 'Dave', 'Eve'}
common = students_A & students_B # {'Bob'}
```

- **Finding Unique Elements:**

```
only_A = students_A - students_B # {'Alice', 'Charlie'}
```

Summary



Operation	Method / Operator	Result
Create	<code>{1, 2, 3} / set()</code>	A set
Add element	<code>s.add(x)</code>	s updated
Add multiple	<code>s.update(iterable)</code>	s updated
Remove (error if missing)	<code>s.remove(x)</code>	s updated
Remove (safe)	<code>s.discard(x)</code>	s updated
Pop arbitrary	<code>s.pop()</code>	Returns element
Clear all	<code>s.clear()</code>	Empty set
Union	<code>s t / s.union(t)</code>	Combined set
Intersection	<code>s & t / s.intersection(t)</code>	Common elements
Difference	<code>s - t / s.difference(t)</code>	In s, not in t
Symmetric Diff	<code>s ^ t / s.symmetric...</code>	Either, not both

Summary



Operation	Method / Operator	Result
Subset	<code>s <= t</code> / <code>s.issubset(t)</code>	True / False
Superset	<code>s >= t</code> / <code>s.issuperset(t)</code>	True / False
Disjoint	<code>s.isdisjoint(t)</code>	True / False
Length	<code>len(s)</code>	Number of items
Membership	<code>x in s</code>	True / False

Summary



- **Sets are unordered, mutable collections of unique elements.**
- **Created with `{ }` (non-empty) or `set()` constructor.**
- **Core operations: Union (`|`), Intersection (`&`), Difference (`-`), Symmetric Difference (`^`), Subset (`<=`), Superset (`>=`).**
- **Methods for modification: `add()`, `update()`, `remove()`, `discard()`, `pop()`, `clear()`, and in-place update variants.**
- **`frozenset` provides an immutable version of sets.**